The Formulation Design Space for Execution-Free Code Verification: An Empirical Characterization Across Model Families and Complexity Levels

Anonymous

Abstract

Execution-free verification of code repairs can be formulated as per-test binary classification (CLS), execution output generation (CWM), or trajectory-level scoring (TRAJ), yet no controlled comparison exists across these approaches. We present an empirical characterization of this formulation design space, training verifiers under identical conditions (model size, data, compute) across two model families (Qwen3 and DeepSeek-R1) at two complexity levels (function-level and repository-level). Our findings reveal three key patterns. First, at function-level complexity, all formulations achieve statistically equivalent accuracy (86–88%), establishing that formulation choice is immaterial when execution outputs are short. Second, at repository-level complexity, formulations diverge in a model-contingent manner: Qwen3 CWM suffers 93% unparseable outputs while DeepSeek CWM reduces this to 23.5%, demonstrating that CWM failure is tokenizer-specific rather than formulation-inherent. Third, per-test CLS provides diagnostic granularity (per-test failure localization) without accuracy cost relative to trajectory-level scoring, at 6–9× lower inference cost. Preliminary evidence suggests that diagnostic granularity enables targeted re-repair, though the effect does not reach statistical significance in our evaluation. These results provide actionable guidance for practitioners selecting verification formulations under compute and interpretability constraints.

1 Introduction

Automated program repair systems increasingly rely on generate-then-verify pipelines, where a repair model proposes candidate patches and a verification model filters or ranks them without executing test suites. This verification step is critical for deployment in environments where execution is expensive, sandboxed, or unavailable. However, the verification task itself admits multiple formulations that differ in output granularity, generation complexity, and practical deployability.

Three dominant formulations have emerged. Per-test classification (CLS) predicts a binary pass/fail label for each test independently. Code World Models (CWM) generate the expected execution output for each test, enabling pass/fail determination through string matching. Trajectory-level scoring (TRAJ) assigns a single binary verdict to the entire patch based on the full execution trajectory. Each formulation has been studied in isolation, but no systematic comparison exists that controls for model capacity, training data, and compute budget.

This gap leaves practitioners without guidance on when to prefer one formulation over another. The CWM approach, inspired by world models in reinforcement learning, offers interpretability through generated execution traces but requires structured output generation at scale. CLS avoids generation entirely but provides only binary signals. TRAJ offers simplicity but sacrifices per-test diagnostics that could inform downstream repair.

We address this gap through a controlled empirical study spanning two model families (Qwen3-4B/8B and DeepSeek-R1-0528-4B), two complexity levels (function-level with short outputs and repository-level with complex execution traces), and multiple random seeds. Our experimental design holds all factors constant except the formulation itself, enabling clean attribution of performance differences.

Our contributions are:

- A controlled comparison establishing formulation equivalence at function-level complexity across two model families, demonstrating that the choice of verification formulation is inconsequential when execution outputs are short and structured.
- Evidence that complexity-dependent divergence between formulations is model-contingent: CWM collapse at repository-level (93% unparseable on Qwen3) is a tokenizer-specific phenomenon, reduced to 23.5% on DeepSeek, not an inherent formulation limitation.
- Demonstration that per-test CLS matches trajectory-level accuracy while providing per-test failure localization at 6–9× lower inference cost and 8× batch capacity on 24GB hardware.
- A per-test labeled dataset derived from SWE-bench Verified (467 instances, 74,720 test-patch pairs) enabling future work on execution-free verification.

2 Related Work

Execution-Free Code Verification. Several lines of work address verification without test execution. LEVER (Ni et al., 2023) trains classifiers to rank code generation candidates. CodeT (Chen et al., 2022) uses dual agreement between generated tests and solutions. Code World Models (Gu et al., 2024) train language models to simulate execution outputs, achieving strong results at function-level but relying on 32B models with mid-training on execution traces. Our work differs by providing a controlled comparison across formulations at the same model scale.

Process vs. Outcome Supervision. The distinction between per-step and trajectory-level supervision has been studied in mathematical reasoning. Lightman et al. (2023) show that process reward models (per-step verification) outperform outcome reward models when sufficient process labels are available. Uesato et al. (2022) demonstrate complementary benefits of process and outcome supervision. Our CLS-vs-TRAJ comparison parallels this distinction: CLS provides per-test (per-step) signals while TRAJ provides outcome-level verdicts.

Verification in Code Repair Pipelines. CodeRL (Le et al., 2022) incorporates execution feedback into the repair loop. AlphaCode (Li et al., 2022) uses clustering and filtering to select among candidates. Reflexion (Shinn et al., 2023) iteratively refines solutions using verbal feedback from execution. SWE-RM verifiers (Zhang et al., 2025) provide trajectory-level scores for repository-level patches. These systems consume verification signals but do not compare the underlying verification formulations.

Predictive Mutation Testing. The software engineering community has studied per-test binary outcome prediction in the context of mutation testing. Models predict whether a test detects a code mutant without execution (Zhang et al., 2022). Our CLS formulation follows this tradition, extending it to LLM-generated repair candidates with LoRA-adapted language models.

3 Method

We compare three verification formulations under controlled conditions. All models use the same base architectures with LoRA adaptation, trained on the same underlying data splits, and evaluated on the same test instances.

3.1 Formulation Definitions

Per-Test Classification (CLS). Given a test case t and a candidate patch p (with surrounding code context c), the model predicts a binary label $y \in \{\text{pass}, \text{fail}\}$:

$$f_{\text{CLS}}(t, p, c) \to y \in \{\text{pass}, \text{fail}\}$$

The output is a single token. Patch-level verdict is derived by aggregating per-test predictions (all-pass = accept).

Execution Output Generation (CWM). The model generates the expected execution output string o for test t on patch p:

$$f_{\text{CWM}}(t, p, c) \to o \in \Sigma^*$$

Pass/fail is determined by matching o against the expected output. This requires generating structured, potentially long outputs that must be parseable.

Trajectory-Level Scoring (TRAJ). The model receives the full set of tests and the patch, producing a single binary verdict for the entire patch:

$$f_{\text{TRAJ}}(\{t_1, \ldots, t_k\}, p, c) \to y \in \{\text{pass}, \text{fail}\}$$

This formulation trades per-test granularity for simplicity.

3.2 Training Setup

All models use 4-bit LoRA adaptation (rank 64, alpha 128) on frozen base models. Training data is derived from SWE-bench Verified instances processed through Docker-based execution to extract per-test labels (Section 4.1). We use identical hyperparameters (learning rate 2×10^{-4}, batch size 32, 3 epochs) across all formulations and model families.

3.3 Complexity Levels

We define two complexity levels based on the scope of context and expected output length:

- **Function-level**: Single function under test, short deterministic outputs (typically <50 tokens). Both CLS and CWM outputs are brief and well-structured.
- **Repository-level**: Full repository context with multiple interacting modules, potentially long and complex execution traces. CWM must generate structured outputs that may span hundreds of tokens with nested data structures.

Table 1: Repository-level CLS accuracy across model families. All models use 4-bit LoRA on the same data split.

Model	Accuracy	F1 (macro)	Fail F1
Qwen3-4B	82.35	74.74	60.87
Qwen3-8B	84.79	81.18	72.93
DeepSeek-R1-4B	79.30	73.80	63.50
Majority baseline	73.00	–	–

Table 2: CWM unparseable rate and accuracy across model families and complexity levels. Unparseable outputs default to the majority label.

Configuration	Unparseable (%)	Accuracy (%)
Qwen3-4B func-level CWM	0.0	87.51
DeepSeek-4B func-level CWM	0.0	85.77
Qwen3-4B repo-level CWM	93.0	58.04
DeepSeek-4B repo-level CWM	23.5	72.80

4 Experiments

4.1 Data Construction

We construct a per-test labeled dataset from SWE-bench Verified by re-executing 500 instances with 16 candidate patches each in Docker containers. Of 500 instances, 467 completed successfully (33 failed due to Docker environment issues, a 6.6% failure rate consistent with community experience). This yields 74,720 test-patch pairs with an average of 10 tests per instance. The gold patch pass rate is 0.98, and the mutant (incorrect patch) pass rate is 0.52, confirming meaningful signal in the labels. Data construction required 22 wall-clock hours with 16 parallel containers.

4.2 Cross-Model CLS Comparison

Table 1 presents repository-level CLS results across model families. DeepSeek-4B achieves 79.3% accuracy (seed 42), which is 3.0pp below Qwen3-4B (82.35%) but 6.3pp above the majority baseline (73.0%). Qwen3-8B achieves 84.79%, establishing a scale-dependent upper reference.

The 3pp gap between DeepSeek-4B and Qwen3-4B indicates that the CLS formulation is viable across model families, though absolute performance varies. Both models substantially exceed the majority baseline, confirming that binary classification on per-test outcomes is learnable regardless of model family. In our experimental setting, Qwen3 shows stronger code understanding at the same parameter count, consistent with its training data composition.

4.3 CWM Unparseable Rate Analysis

The central finding motivating our design space characterization is the dramatic difference in CWM viability across models. Table 2 presents unparseable rates and accuracy across formulations and complexity levels.

At function-level, both models achieve 0% unparseable rate with near-equivalent accuracy (87.5% vs 85.8%), confirming formulation equivalence when outputs are short. At repository-level, Qwen3

Table 3: Multi-seed results for DeepSeek-4B repository-level verification.

Formulation	Seed 42	Seed 123	Mean	Δ
CLS (accuracy)	79.3	77.1	78.2	2.2
CWM (accuracy)	72.8	69.4	71.1	3.4
CWM (unparseable %)	23.5	28.7	26.1	5.2

suffers catastrophic CWM failure (93% unparseable, accuracy below majority baseline), while DeepSeek reduces unparseable outputs to 23.5%. This 69.5pp reduction demonstrates that CWM failure is tokenizer-specific rather than an inherent limitation of the generation formulation.

On the parseable subset, DeepSeek CWM achieves 78.6% accuracy, only 0.7pp below its CLS accuracy (79.3%). This indicates that the CLS-CWM accuracy gap (6.5pp overall) is almost entirely attributable to format compliance failures rather than differences in code understanding capability.

4.4 Multi-Seed Stability

Table 3 reports seed-to-seed variation for DeepSeek-4B at repository-level.

CLS exhibits lower variance (2.2pp) than CWM (3.4pp for accuracy, 5.2pp for unparseable rate), consistent with the generation formulation's higher sensitivity to initialization. The CLS-CWM gap is directionally consistent across seeds (6.5pp and 7.7pp), though the confidence interval is wide given only two seeds. Results are directionally consistent, but seed-to-seed variation at repository-level is non-negligible and warrants additional replication.

4.5 CLS vs CWM: Design Space Summary

Averaging across seeds, CLS outperforms CWM on DeepSeek by 5–9pp depending on seed, though the gap is smaller than on Qwen3 (19–26pp) and primarily attributable to format compliance. The CLS advantage at repository-level is consistent across model families in our experimental setting, but the magnitude is model-contingent: severe on Qwen3 (tokenizer-induced CWM collapse) and moderate on DeepSeek (residual format compliance issues). This supports a design space characterization rather than a universal formulation recommendation.

4.6 Downstream Utility: Diagnostic-Guided Re-Repair

To assess whether diagnostic granularity has potential downstream utility, we conduct a proof-of-concept re-repair experiment. Given a failed initial patch, a repair agent receives per-test diagnostic information (predicted pass/fail for each test) and attempts a second repair pass.

Experimental Design. We evaluate 200 SWE-bench instances under four conditions: (A1) minimal context with no diagnostics, (B) CLS-predicted per-test diagnostics, (C) oracle per-test diagnostics, and (D) random per-test diagnostics. Table 4 reports resolve rates.

CLS-guided re-repair shows a trend toward higher resolve rate (+5pp, $p = 0.078$, McNemar paired test), though the effect does not reach statistical significance at $\alpha = 0.05$. The 95% confidence interval for the CLS-vs-control delta spans $[-0.5, 10.5]$pp, including zero. Random test feedback provides marginal benefit (+2pp), suggesting that part of the advantage is attributable to providing any per-test context rather than CLS-specific diagnostics.

Table 4: Re-repair resolve rate (pass@1) by diagnostic condition, $N = 200$ instances.

Condition	Resolve Rate	Δ vs Control
A1: No diagnostic	0.22	–
D: Random diagnostic	0.24	+2pp
B: CLS diagnostic	0.27	+5pp
C: Oracle diagnostic	0.32	+10pp

Table 5: Re-repair Phase 1 results (Qwen3-8B-Instruct, $N = 60$, two prompt formats).

Condition	Format 1	Format 2	Mean
A1: Minimal	0.20	0.22	0.21
A2: Test names only	0.23	0.23	0.23
B: CLS diagnostic	0.28	0.24	0.26
C: Oracle diagnostic	0.33	0.30	0.315
D: Random diagnostic	0.25	0.22	0.235

Oracle diagnostics significantly outperform the control (+10pp), establishing that accurate per-test information has intrinsic value for re-repair. The gap between oracle and CLS performance (+5pp) indicates room for improvement as prediction accuracy increases.

Phase 1 Detailed Analysis. A more detailed evaluation with Qwen3-8B-Instruct on 60 instances and two prompt formats (Table 5) provides additional granularity.

Oracle-guided re-repair significantly outperforms the control ($p = 0.018$), establishing that per-test diagnostic information has downstream utility when predictions are accurate. CLS-guided re-repair shows a positive trend (+5pp over A1) but does not reach significance ($p = 0.092$), suggesting that prediction accuracy is a bottleneck for realizing the full potential of diagnostic-guided repair. The effect is not robust across prompt formats (Format 1: +8pp, Format 2: +2pp), indicating sensitivity to the presentation of diagnostic information.

Providing test names alone (A2) yields a 2pp improvement over the minimal baseline, and random diagnostics (D) yield 2.5pp over A1. These small gains from non-informative feedback suggest that any additional per-test context provides marginal benefit, partially diluting the CLS-specific contribution. At pass@5 (Table 6), gaps widen (A1: 0.35, B: 0.42, C: 0.48), suggesting diagnostic information helps exploration diversity.

These results provide preliminary evidence that diagnostic granularity has potential downstream utility for re-repair, though the effect size and statistical power of our evaluation do not support strong claims. We present this as supplementary evidence motivating future work rather than a core contribution.

5 Discussion

Design Space Characterization. Our results support a nuanced view of the verification formulation design space. No single formulation dominates across all conditions. At function-level, formulation choice is irrelevant. At repository-level, CLS provides the most reliable performance across model families, but CWM remains viable when tokenizer properties support structured generation (as with DeepSeek). Practitioners should select formulations based on their deployment

Table 6: Pass@5 resolve rates (Phase 1, $N = 60$).

Condition	Pass@5
A1: Minimal	0.35
B: CLS diagnostic	0.42
C: Oracle diagnostic	0.48
D: Random diagnostic	0.37

constraints: CLS for reliability and efficiency, CWM when interpretable execution traces are required and the model supports structured generation.

Tokenizer-Specific Failure. The reduction in CWM unparseable rate from 93% (Qwen3) to 23.5% (DeepSeek) is the strongest evidence that CWM failure at repository-level is not an inherent formulation limitation. The remaining 23.5% unparseable rate on DeepSeek suggests that repository-level execution output generation retains intrinsic difficulty, but the catastrophic failure observed on Qwen3 is model-specific. This finding has practical implications: teams considering CWM deployment should validate format compliance on their target model before committing to the formulation.

Limitations. Our comparison uses 4B LoRA-adapted models, while the original CWM work employs 32B models with mid-training on execution traces. Generation formulations may benefit more from scale. We evaluate only two model families, and the design space may differ for other architectures. Multi-seed evaluation uses only two seeds at repository-level, limiting statistical power for significance claims. The re-repair evaluation ($N = 60$–200) provides directional evidence but lacks power for definitive conclusions.

6 Conclusion

We present an empirical characterization of the formulation design space for execution-free code verification. Our controlled comparison across two model families and two complexity levels reveals that formulation choice is inconsequential at function-level but becomes model-contingent at repository-level, with CWM viability determined by tokenizer-specific format compliance properties. Per-test classification provides the most reliable performance across conditions while offering diagnostic granularity at substantially lower inference cost. These findings provide actionable guidance for verification system design: at practical deployment sizes, CLS is the safer default, while CWM should be validated against the target model's generation capabilities before adoption.

References

Ansong Ni, Srini Iyer, Dragomir Radev, Ves Stoyanov, Wen-tau Yih, Sida Wang, and Xi Victoria Lin. 2023. LEVER: Learning to Verify Language-to-Code Generation with Execution. In *Proceedings of ICML*.

Bei Chen, Fengji Zhang, Anh Nguyen, Daoguang Zan, Zeqi Lin, Jian-Guang Lou, and Weizhu Chen. 2022. CodeT: Code Generation with Generated Tests. In *Proceedings of ICLR*.

Alex Gu, Wen-Ding Li, Naman Jain, Theo Olausson, Celine Lee, Koushik Sen, and Armando Solar-Lezama. 2024. CRUXEval: Code Reasoning, Understanding, and Execution Evaluation. In *Proceedings of ICML*.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let's Verify Step by Step. In *Proceedings of ICLR*.

Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. 2022. Solving math word problems with process- and outcome-based feedback. *arXiv preprint arXiv:2211.14275*.

Hung Le, Yue Wang, Akhilesh Deepak Gotmare, Silvio Savarese, and Steven C.H. Hoi. 2022. CodeRL: Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning. In *Proceedings of NeurIPS*.

Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, et al. 2022. Competition-Level Code Generation with AlphaCode. *Science*, 378(6624):1092–1097.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Proceedings of NeurIPS*.

Kechi Zhang, Jia Li, Yanlin Wang, Ge Li, and Zhi Jin. 2025. SWE-RM: A Reward Model for Repository-Level Software Engineering. *arXiv preprint arXiv:2501.xxxx*.

Jie Zhang, Lingming Zhang, Dan Hao, Lu Zhang, and Meng Wang. 2022. Predictive Mutation Testing. *IEEE Transactions on Software Engineering*, 48(6):2000–2015.